Spring 2026 Analytics II Team Project Proposal Report

Group 1

Joshua Boyles
Dave Julianto
Hayden Burke
Daniel Veiner
Sebastian Hajdu

Professor: Dr. Kaixun Hua

Abstract

Briefly summarize the problem, the proposed method, and the main findings. Keep it concise (typically 150–250 words). Daniel volunteered to do this section.

1 Introduction

This project aims to utilize historical ball bearing manufacturing data to predict defectiveness of manufactured parts and to understand which combinations of control settings typically lead to defective production. The goal is to create an interpretable model that accurately predicts whether a ball bearing will be classified as defective after several destructive and non-destructive tests, based on a set of predictor variables.

The dataset contains approximately 230,000 samples with 30 predictor variables. Each sample is labeled as defective (class 1) or non-defective (class 0), which is the target variable we aim to predict. By understanding the factors that contribute to defects, corrective action can be taken on controllable system inputs to reduce future defects. Reduced defect rates directly lower scrap, rework, resource, and labor costs in the manufacturing process.

However, this problem presents several challenges: the dataset is large and high-dimensional, the classes may be imbalanced (with relatively few defective samples), and manufacturing processes can involve complex, nonlinear interactions between variables. Additionally, interpretability is critical, as predictions must provide actionable insights for engineers and management.

In this project, we aim to:

1. Develop an interpretable predictive model for classifying defective ball bearings.
2. Identify and analyze key process variables that most strongly influence defect rates.
3. Explore how the model's insights can guide corrective actions to reduce defects and improve manufacturing efficiency.

Together, these aims outline a plan to leverage historical manufacturing data for actionable defect prediction and process improvement.

2 Related Work

Machine learning has increasingly been applied to manufacturing systems for predictive quality monitoring and defect detection. Prior research shows that industrial process and test data can be used to learn relationships between operating conditions and product quality outcomes.

Zahrani et al. apply machine learning models to predict product quality and groove geometry in a circular laser-grooving manufacturing process. Their results show that data-driven models can capture nonlinear relationships between process parameters and final part characteristics, supporting predictive monitoring rather than purely reactive inspection [1].

More recent studies extend this direction using stronger ensemble methods for rare-defect prediction in industrial environments. Geng et al. propose a framework for predicting internal cracks in continuous steel casting using CatBoost combined with synthetic data generation, demonstrating improved detection of infrequent defect patterns in a real manufacturing context [2].

A central challenge in manufacturing defect prediction is severe class imbalance. Defective products may represent only a small fraction of observations, causing standard training and evaluation to favor the majority class and report high accuracy while missing rare failures. Kafunah et al. examine this issue in fault detection and evaluate imbalance-aware strategies such as oversampling and reweighting to improve minority-class detection, highlighting the importance of focusing on defect-class metrics such as precision, recall, and F1 rather than accuracy alone [3].

Cost-sensitive learning provides a complementary approach by explicitly assigning higher penalties to misclassifying defective samples, reflecting the higher operational risk of false negatives in manufacturing. Liu et al. introduce a cost-sensitive defect detection framework that modifies the

training objective using an explicit cost structure to prioritize costly minority-class errors, and they report improved defect identification under imbalance [4].

Building on these findings, our project systematically compares interpretable and ensemble models, including logistic regression, decision trees, random forests, and boosting under multiple imbalance-mitigation strategies. We emphasize cost-sensitive random forests and minority-class performance metrics to evaluate model effectiveness for defect detection in a highly imbalanced manufacturing dataset.

3 Model and Method

Class Imbalance - Preferred Metrics

The dataset comprises 229,788 ball-bearing test records featuring 30 numerical predictors and a binary response variable. With only 458 defective samples, standard classifiers tend to favor the majority class; yielding high accuracy that can be misleading. In manufacturing, false negatives are more costly than false positives, as they can lead to diminished customer confidence, expensive product recalls, or liability lawsuits. Consequently, accuracy is not an appropriate metric. Instead, recall, precision, F1 Score, and the Area Under the Precision-Recall Curve (PR-AUC) are the primary metrics.

Because the defective class represents only 0.2% of the dataset, using Receiver Operating Characteristic Area Under Curve (ROC AUC) can overly estimate the performance. A study was conducted and proved that ROC AUC is not dependable if the true positive sample is less than 3% of the total [5]. PR AUC is preferred because it summarizes the precision and recall tradeoff across thresholds and reflects performance exclusively on the defective class, suitable for the case of missing a defect is far more consequential than flagging a non defective part. However, ROC AUC could still be considered to gauge the performance of baseline models.

Using regular F1 score as a metric combines performance across all classes, which causes the majority class to dominate the result in imbalanced datasets, shown by this formula:

$$F1_{regular} = 2 \cdot \frac{P_{overall} \cdot R_{overall}}{P_{overall} + R_{overall}}$$

where $P_{overall}$ and $R_{overall}$ are precision and recall across both classes. Because these aggregated values are driven primarily by the majority class, a model can achieve a high regular F1 score even while performing poorly on the defective class. F1 per class, especially F1 minority avoids this masking effect by computing precision and recall only for the defective class:

$$F1_{minority} = 2 \cdot \frac{P_{minority} \cdot R_{minority}}{P_{minority} + R_{minority}}$$

The criticality of using F1 to detect the minority class in classification serves to aid in anomaly detection and identify the imbalance of the data set and its analysis [6]. Classification trained on imbalanced data is weak at predicting samples within the minority class.

Class Imbalance - Modification on Training Data

To address the imbalance, four manipulation strategies are employed, each with its distinct trade-offs. Random Oversampling duplicates minority samples to improve recall, though it risks overfitting. Random Undersampling removes majority samples to balance the classes but may lead to information loss. Synthetic Minority Oversampling Technique (SMOTE) generates synthetic defective samples by interpolation to refine decision boundaries but may introduce noise if the minority class has high variance. Finally, Class-Weighted training adjusts the learning objective to penalize minority-class errors more heavily, shifting the decision boundary without altering the dataset. These methods are systematically evaluated by comparing performance metrics across the experimental design combinations.

Model Considerations

We plan to evaluate four primary models, which are logistic regression, Decision Tree, Random Forest, and AdaBoost/XGBoost. By comparing these simple and complex models, performance trade-offs between interpretability and prediction results can be measured.

Logistic Regression

Logistic Regression is a sensible model to explore as it is suitable for binary classification problems. Logistic Regression models the probability that a bearing is defective as a smooth function of the readings:

$$P(defective|x) = \sigma(\beta_0 + \beta^\top x) = \frac{1}{1 + e^{-(\beta_0 + \beta^\top x)}}$$

Here, x is the vector of reading values, β is the vector of coefficients, and $\sigma(\cdot)$ is the logistic (sigmoid) function that maps any real number to a probability between 0 and 1. Each coefficient β_i represents the change in the log-odds of a defect associated with a one-unit increase in reading x_i, holding other predictors constant. This makes Logistic Regression highly interpretable as engineers can directly see which predictors increase or decrease defect risk and by how much.

Decision Trees

Decision Trees provide transparency by sorting data through a series of logical, binary splits. At each step, the model identifies the sensor threshold that best separates defective bearings from healthy ones, reducing impurity until a homogeneous classification is reached. This visual approach allows us to see exactly which predictor values trigger a defect alert. To determine the best possible split, the algorithm calculates a purity score at each step. This is done using Gini Impurity, which measures the probability of a specific sensor reading being incorrectly classified if it were labeled randomly. A lower Gini value indicates a more pure or successful separation of defective bearings from healthy ones. Gini is calculated as follows:

$$Gini(S) = \sum_{k \in \{0,1\}} p_k(1 - p_k)$$

where p_k is the proportion of class k in node S. A split into left and right child nodes S_L and S_R is evaluated by:

$$\Delta = Gini(S) - \left(\frac{|S_L|}{|S|} Gini(S_L) + \frac{|S_R|}{|S|} Gini(S_R) \right)$$

The algorithm selects the split with the largest Δ, the greatest reduction in impurity. Gini impurity is used instead of entropy because it is computationally simpler by not using logarithms and tends to be more numerically stable under extreme class imbalance. As the defective class is severely smaller, tiny fluctuations in estimated probabilities can cause large changes in entropy, potentially amplifying noise. Gini provides smoother behavior, leading to more stable splits and more reliable rankings of feature importance. For this project, Decision Tree models serve as an interpretable nonlinear model that can capture simple interactions between predictors, and as a base learner for ensemble methods.

Random Forest

Random Forest extends Decision Trees by aggregating many trees trained on different subsets of the data and features. Each tree is trained on a bootstrap sample of the training data, and at each split, a random subset of features is considered. The final prediction is obtained by majority vote:

$$\hat{y} = mode\{\hat{y}^{(1)}, \hat{y}^{(2)}, \ldots, \hat{y}^{(N)}\}$$

where $\hat{y}^{(n)}$ is the prediction of tree n, and N is the number of trees. The ensemble model determines the best prediction by calculating the mode across the results of each individual tree. To address class imbalance, the Balanced Random Forest variant is used: for each tree, all defective samples are included, and an equal number of non-defective samples are randomly drawn to form a balanced bootstrap sample. This ensures that each tree is trained on a dataset where the defective class is not overwhelmed by the majority class, improving the model's ability to learn splits that separate defects from non-defects and thereby improving recall. Random Forests also provide feature importance

scores (e.g., based on impurity reduction or permutation importance), which are valuable for manufacturing engineers who need to understand which sensors consistently contribute to defect detection across many trees.

Boosting

Adaptive Boosting (AdaBoost) builds an ensemble of weak learners by repeatedly reweighting the training samples so that each new classifier focuses on the mistakes of the previous ones. The misclassified samples get their weights w_j^i increased, correctly classified ones decreased, and then they all get normalized by the normalization constant. Over time, this forces the ensemble to concentrate on high-error instances:

$$\epsilon_i = \sum_j w_j^i \delta(C_i(x_j) \neq y_j)$$

This quantity measures how much weight the current classifier assigns to its mistakes. The classifier's importance weight is then computed as:

$$\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \epsilon_t}{\epsilon_t} \right)$$

The importance increases when the classifier performs better than chance and decreases when it performs poorly. After each iteration, AdaBoost updates the sample weights:

$$w_j^{i+1} = \frac{w_j^l}{Z_i} \times \begin{cases} e^{-\alpha_i}, & C_i(x_j) = y_j \\ e^{\alpha_i}, & C_i(x_j) \neq y_j \end{cases}$$

The misclassified samples get their weights increased by e^{α_i}, correctly classified ones decreased by $e^{-\alpha_i}$, and then they all get normalized by Z_i, the normalization constant. Over time, this forces the ensemble to concentrate on high error samples. This behavior is useful for classifying rare defective bearings in our data. As they are rare, they are often misclassified by early weak learners because they appear infrequently and have subtle patterns. AdaBoost naturally increases their weight, causing them to gain importance. This can improve defect detection without manually oversampling. However, AdaBoost is sensitive to noise; overemphasizing and overfitting if there is too much. For this reason, AdaBoost serves as a strong but potentially sensitive benchmark in the project timeline.

Project Timeline

The following procedure outlines the structured progression from data preparation to advanced ensemble modeling, ensuring that the extreme class imbalance is systematically addressed.

1. **Data Inspection (1–2 hours):** Load the dataset and analyze the distribution.
2. **Exploratory Data Analysis (4–6 hours):** Identify sensor ranges, correlations, and outliers across the 30 numerical predictors to understand baseline behavior.
3. **Partitioning (1–2 hours):** Split data into training (80%), validation (10%), and test sets (10%).
4. **Imbalance Handling (3–5 hours):** Apply strategies including Random Oversampling, Undersampling, SMOTE, and Class-Weighted training within cross-validation folds to prevent data leakage.
5. **Simple Modeling (4–6 hours):** Train Logistic Regression and Decision Trees to establish interpretable reference performance and identify primary predictors.
6. **Ensemble Modeling (8–12 hours):** Train Random Forest (using the Balanced variant) and Boosting methods (AdaBoost/XGBoost) to capture complex non-linear interactions.
7. **Performance Evaluation (3–5 hours):** Assess all models using metrics that reflect the high cost of false negatives: Precision, Recall, F1-minority, and PR-AUC.
8. **Ablation Study (6–8 hours):** Conduct a systematic comparison by "swapping out" imbalance strategies to identify which method provides the most stable defect detection.
9. **Interpretability Analysis (4–6 hours):** Research and utilize SHAP values and permutation importance to rank the most influential sensors and explain model decisions to maintenance teams.

Table 1: Estimated Time Allocation per Phase

Phase	Estimated Hours
Data Inspection	1–2
Exploratory Data Analysis	4–6
Partitioning	1–2
Imbalance Handling	3–5
Simple Modeling	4–6
Ensemble Modeling	8–12
Performance Evaluation	3–5
Ablation Study	6–8
Interpretability Analysis	4–6
Total Estimated Time	**24–38 hours**

Experimental Design

The experimental phase is structured into three distinct components to ensure a robust evaluation of the model.

- **Model Benchmark:** A comparison of simple and complex models to determine the best performing model; with consideration of interpretability vs. predictive performance.
- **Class Imbalance Strategy Ablation:** A systematic evaluation of various dataset manipulation strategies to address the class imbalance problem.
- **Data Split Comparison:** A test of model stability and generalization across different training, validation, and test ratios.

Main Benchmark Analysis

Objective: Evaluate candidate models under realistic manufacturing constraints to identify the optimal balance between predictive power and model transparency. This benchmark compares the interpretability vs. performance of models and is often as critical as its accuracy in industrial maintenance environments. We prioritize Recall and F1-minority to ensure rare defects are not overlooked.

Evaluation Focus:

- **Minority Class Sensitivity:** Prioritizing **Recall** and **F1-minority** to ensure rare defects are not overlooked.
- **Metric Integrity:** Using **PR-AUC** to provide a truthful assessment, as **ROC-AUC** tends to be overly optimistic when the minority class is <3% [5].
- **Architectural Complexity:** Quantifying the "cost" of transparency using specific indicators:
  - **Linear Models:** Number of non-zero coefficients (Sparsity).
  - **Tree Models:** Maximum tree depth and leaf count.
  - **Ensembles:** Number of estimators and feature importance stability.
- **Economic Impact:** Tracking **False Negatives (FN)** as a direct proxy for the cost of manufacturing failure.

Model Family	Recall	F1-Min.	PR-AUC	ROC-AUC	Complexity Indicator	FN	FP
Logistic Regression					# Coefficients		
Decision Tree					Tree Depth		
Balanced RF					# Trees / Depth		
XGBoost/LightGBM					# Estimators		

Table 2: Benchmark Results Placeholder

Expected Outcomes:

- **Superiority of Ensembles:** Boosting and Forest-based methods are expected to yield the highest PR-AUC, revealing significant performance gaps that ROC-AUC may obscure.
- **Diminishing Returns:** We anticipate a curve where complex boosting methods provide only marginal gains over Balanced Random Forests, but at the cost of significantly higher complexity and potential brittleness to noisy minority samples.
- **Interpretability:** Baseline models will likely provide the highest transparency but lower recall, serving as the essential "control group" for the study.

Class Imbalance Strategy Ablation

Objective: Quantify the impact of various sampling and weighting strategies on the decision boundary. Each of the strategies will be tested to each of the model selection, creating 16 experimental combinations.

- **Oversampling/SMOTE:** Testing if synthetic minority generation improves recall without introducing excessive noise.
- **Undersampling:** Assessing if reducing the majority class leads to a critical loss of "normal" state information.
- **Cost-Sensitive Learning:** Evaluating if class weighting offers a cleaner alternative to data manipulation.

Expected Outcome:

Class weighting and Balanced Random Forest (internal undersampling) are expected to provide the most stable results, whereas SMOTE may increase False Positives by creating unrealistic defective signatures.

Data Split Comparison

This test is to Determine if the model's performance is an artifact of a specific data split, especially given the limited minority sample size. The baseline data split will use 80/10/10 of training, validation, and test data, respectively.

Split Ratio	Recall	Precision	F1-Minority	PR-AUC	FN	FP
70/15/15						
80/10/10						
90/05/05						

Table 3: Model Stability Across Split Ratios

References

[1] E. G. Zahrani, F. Hojati, A. Daneshi, B. Azarhoushang, and J. Wilde. Application of machine learning to predict the product quality and geometry in circular laser grooving process. *Procedia CIRP*, 94:474–480, 2020.

[2] M. Geng, H. Ma, S. Liu, Z. Zhou, L. Xing, Y. Ai, and W. Zhang. A data-driven approach for internal crack prediction in continuous casting of HSLA steels using CTGAN and CatBoost. *Materials*, 18:3599, 2025.

[3] J. Kafunah, M. I. Ali, and J. G. Breslin. Handling imbalanced datasets for robust deep neural network-based fault detection in manufacturing systems. *Applied Sciences*, 11:9783, 2021.

[4] B. Liu, F. Gao, and Y. Li. Cost-sensitive YOLOv5 for detecting surface defects of industrial products. *Sensors*, 23:2610, 2023.

[5] M. Imani, M. Joudaki, A. Bagheri, and H. R. Arabnia. Why ROC-AUC is misleading for highly imbalanced data: In-depth evaluation of MCC, F2-Score, H-Measure, and AUC-based metrics across diverse classifiers. *Technologies*, 14(1):54, 2026.

[6] ScienceDirect. Minority class. *ScienceDirect Topics*. Retrieved March 5, 2025.

Appendix: Python Implementation

```
import pandas as pd                                     # Import
from sklearn.model_selection import train_test_split
from imblearn.pipeline import Pipeline
from imblearn.over_sampling import SMOTE
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier
from sklearn.metrics import classification_report, average_precision_score
import shap

# --- PART 1: STRATIFIED DATA SPLIT ---
# Preserves the 0.2% minority class ratio across splits
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, stratify=y,
random_state=42)

# --- PART 2: MODEL INITIALIZATION ---
# Define the models to be evaluated in the pipeline
models = {"Logistic Regression": LogisticRegression(class_weight='balanced',
max_iter=1000),
    "Decision Tree": DecisionTreeClassifier(class_weight='balanced'),
    "Random Forest": RandomForestClassifier(n_estimators=100,
class_weight='balanced_subsample'),
    "AdaBoost": AdaBoostClassifier(n_estimators=100)}

# --- PART 3: PIPELINE EXECUTION (SMOTE + MODEL) ---
# Wrapping the sampler and model in a Pipeline prevents data leakage
for name, model in models.items():
    pipeline = Pipeline([
        ('sampler', SMOTE(random_state=42)),            # Imbalance Strategy
        ('classifier', model)])                         # The Model
    pipeline.fit(X_train, y_train)                      # Train
    y_pred = pipeline.predict(X_test)                   # Predict
    y_probs = pipeline.predict_proba(X_test)[:, 1]

# --- PART 4: EVALUATION ---
    print(f"REPORT FOR {name}:")
    print(classification_report(y_test, y_pred))
    print(f"PR-AUC Score: {average_precision_score(y_test, y_probs):.4f}\n")

# --- PART 5: INTERPRETABILITY (SHAP Example for Random Forest) ---
# Interpreting which sensors influence the defective class
rf_model = models["Random Forest"].fit(X_train, y_train)
explainer = shap.TreeExplainer(rf_model)
shap_values = explainer.shap_values(X_test)
shap.summary_plot(shap_values, X_test)
```

Project Progress Tracking

Table 1:

#	Task Description	Est. Hours	Act. Hours	Status
1	Data Loading & Inspection	1-2		
2	Exploratory Data Analysis (EDA)	4-6		
3	Stratified Data Partitioning	1-2		
4	Imbalance Handling Implementation	3-5		
5	Baseline Model Training	4-6		
6	Advanced Model Training (Ensembles)	8-12		
7	Performance Evaluation & Metrics	3-5		
8	Ablation Study Comparison	6-8		
9	SHAP & Interpretability Analysis	4-6		